

Türme von Hanoi

Allgemeine Hinweise

- Erstellen Sie alle Klassen im Paket `de.uniwue.jpp.hanoiTowers` .
- Das Paket befindet sich im Ordner `src/main/java` .
- Es ist gestattet, aber nicht nötig, neben den geforderten Klassen weitere Klassen oder zusätzliche Methoden zu implementieren.

Unit-Tests

- Es stehen lokale Unit-Tests zum Überprüfen Ihrer Implementierung zur Verfügung.
- Die Testklassen befinden sich im Ordner `src/test/java` .
- Jede Aufgabe in dieser Aufgabenstellung hat eine passende Testklasse (Name: "Test[Klassenname]"). Ihre zur Aufgabe gehörigen Klassen müssen kompilieren, damit der Test ausführbar ist.

Aufgaben

Die Türme von Hanoi sind ein mathematisches Knobel- und Geduldsspiel.

Es besteht aus drei Türmen auf die Scheiben verschiedener Größe gelegt werden können. Zu Beginn sind alle Scheiben auf einem Turm der Größe nach geordnet mit der größten Scheibe unten und der kleinsten oben. Bei jedem Zug darf die oberste Scheibe eines beliebigen Feldes auf eines der beiden anderen Felder gelegt werden, vorausgesetzt dort liegt nicht schon eine kleinere Scheibe. Ziel des Spiels ist es, den kompletten Scheibenstapel auf einen anderen Turm zu versetzen.

Vermutlich wurde das Spiel 1883 vom französischen Mathematiker Edouard Lucas erfunden. Er dachte sich dazu die Geschichte aus, dass indische Mönche im großen Tempel zu Benares im Mittelpunkt der Welt einen Turm aus 64 goldenen Scheiben versetzen müssten und wenn ihnen das gelungen sei, wäre das Ende der Welt gekommen.

Die Klausur ist in die folgenden Aufgaben gegliedert.

Die Angaben bezüglich der Punkte (P.) sind ohne Gewähr und dienen der groben Orientierung.

- | | |
|-----------------------------------|---------|
| 1. Die Enum <code>Position</code> | (3 P.) |
| 2. Die Klasse <code>Disk</code> | (12 P.) |
| 3. Die Klasse <code>Move</code> | (10 P.) |
| 4. Die Klasse <code>Tower</code> | (20 P.) |
| 5. Die Klasse <code>Game</code> | (45 P.) |
| | (90 P.) |

Wir wünschen viel Erfolg!

Die Enum `Position`

Implementieren Sie den Aufzählungstyp `Position` mit den Aufzählungen `LEFT` , `MIDDLE` und `RIGHT` . Dieser Aufzählungstyp entspricht den Positionen der Türme.

Zugehörige Testklasse: `PositionTest`

Die Klasse Disk

Implementieren Sie die Klasse `Disk`, die das Interface `Comparable<Disk>` implementiert. Diese Klasse entspricht den einzelnen Scheiben.

Implementieren Sie folgenden Konstruktor:

- `public Disk(int size)`
Erstellt ein neues `Disk`-Objekt mit der übergebenen Größe. Wird eine Größe von 0 oder eine negative Größe übergeben, so ist eine `IllegalArgumentException` mit einer aussagekräftigen Meldung zu werfen.

Implementieren Sie folgende Methoden:

- `public int getSize()`
Liefert die Größe der Disk zurück.

Überschreiben Sie folgende Methoden:

- `public String toString()`
Liefert als Stringrepräsentation die Größe zurück.
- `public boolean equals()`
Zwei Scheiben sind genau dann gleich, wenn Sie die gleiche Größe haben.
- `public int hashCode()`
Folgen Sie hierbei dem in der API vorgegebenen Vertrag zwischen `hashCode` und `equals`.
- `public int compareTo(Disk o)`
Vergleicht die Scheiben so, dass innerhalb einer Liste Scheiben mit einer höheren Größe hinter Scheiben mit einer niedrigeren Größe einsortiert werden.

Zugehörige Testklasse: `DiskTest`

Die Klasse Move

Implementieren Sie die Klasse `Move`. Diese stellt eine Bewegung einer Disk von einem Turm zu einem anderen dar.

Implementieren Sie folgenden Konstruktor:

- `public Move(Position start, Position target, Disk disk)`
Erstellt ein neues `Move`-Objekt mit übergebenem Start, Ziel und Scheibe. Sind Start und Ziel gleich, so ist eine `IllegalArgumentException` mit einer aussagekräftigen Meldung zu werfen.

Implementieren Sie folgende Methoden:

- `public Position getStart()`
Liefert die Ausgangsposition zurück.
- `public Position getTarget()`
Liefert die Zielposition zurück.
- `public Disk getDisk()`
Liefert die Scheibe zurück.

Überschreiben Sie folgende Methoden:

- `public String toString()`
Liefert als Stringrepräsentation zurück: Moving from <Start> to <Ziel> a disk of size <Scheibengröße>.
Beispiel: Moving from LEFT to MIDDLE a disk of size 3.

Zugehörige Testklasse: `MoveTest`

Die Klasse Tower

Implementieren Sie die Klasse `Tower`. Diese stellt einen einzelnen Turm dar.

Implementieren Sie folgende Konstruktoren:

- `public Tower()`
Erstellt eine neues `Tower`-Objekt ohne Scheiben.
- `public Tower(List<Disk> disks)`
Erstellt eine neues `Tower`-Objekt mit den übergebenen Scheiben. Sind die Scheiben nicht absteigend sortiert, so ist eine `IllegalArgumentException` mit einer aussagekräftigen Meldung zu werfen.
- `public Tower(int height)`
Erstellt eine neues `Tower`-Objekt mit `height` Scheiben der Größen 1 bis `height`. Ist `height` negativ, so ist eine `IllegalArgumentException` mit einer aussagekräftigen Meldung zu werfen.

Implementieren Sie folgende Methoden:

- `public int height()`
Liefert die gröÙe des Stapels zurück..
- `public List<Disk> getDisks()`
Liefert die Scheiben zurück. Über das zurückgegebenen Objekt soll die interne Datenstruktur nicht manipuliert werden können.
- `public Disk top()`
Liefert die oberste Scheibe zurück. Ist der Turm leer, ist `null` zurückzugeben.
- `public Disk pop()`
Liefert die oberste Scheibe zurück und entfernt Sie vom Stapel. Ist der Turm leer und es kann keine Scheibe entfernt werden, so ist `null` zurück zu geben.
- `public boolean pushable(Disk disk)`
Prüft, ob eine `Disk` auf den Stapel gelegt werden kann. Dazu muss sie kleiner sein, als die bisher oberste Scheibe.
- `public boolean push(Disk disk)`
Fügt ein `Disk`-Objekt dem Stapel hinzu. Ist dies erfolgreich ist `true` zurückzugeben, sonst `false`. Bedingungen wie bei `pushable`.

Überschreiben Sie folgende Methoden:

- `public String toString()`
Liefert als Stringrepräsentation die Scheiben von unten nach oben, durch Kommas getrennt zurück.
Beispiel: 5,4,1

Zugehörige Testklasse: `TowerTest`

Die Klasse Game

Implementieren Sie die Klasse `Game`. Diese fügt die bisher erstellten Fragmente zu einem gesamten Spiel zusammen.

Implementieren Sie folgenden Konstruktor:

- `public Game(int heightLeft)`

Erstellt ein neues `Game`-Objekt mit drei Türmen. Der linke Turm soll dabei `heightLeft` Scheiben entsprechend des entsprechenden Konstruktors von `Tower` erhalten, alle anderen Türme sind leer. Ist `heightLeft` gleich 0 oder negativ, so ist eine `IllegalArgumentException` mit einer aussagekräftigen Meldung zu werfen.

Implementieren Sie folgende Methoden:

- `public Tower towerAt(Position position)`

Liefert den Turm an der übergebenen Position zurück.

- `public Collection<Position> freeTowers()`

Liefert alle Positionen zurück, an denen sich leere Türme befinden.

- `public Collection<Position> movePossibleFromTo(Position source)`

Liefert alle Positionen zurück, auf die die oberste Scheibe von `source` verschoben werden kann. Ist der Turm an der Position `source` leer, so ist eine leere `Collection` zurückzugeben.

- `public Move move(Position source, Position target)`

Bewegt die oberste Scheibe von `source` nach `target`. Zusätzlich ist ein `Move`-Objekt, das Quelle und Ziel, sowie die verschobene Scheibe enthält zurückzugeben. Ist ein Zug nicht möglich, so ist das Spielfeld zu belassen und `null` zurückzugeben.

- `public List<Move> solve(Position source, Position intermediate, Position target, int level)`

Verschiebt nach folgendem Algorithmus alle Scheiben vom Start zum Zielstapel. Dabei sind alle Schritte in der ausgeführten Reihenfolge zurückzugeben.

Lösen (Start, Hilfspunkt, Ziel, Level)

Wenn `level = 1`

 Ziehe Start -> Ziel

Sonst

 Lösen (Start, Ziel, Hilfspunkt, Level -1)

 Ziehen Start -> Ziel

 Lösen (Hilfspunkt, Start, Ziel, Level -1)

Überschreiben Sie folgende Methode:

- `public String toString()`

Liefert als Stringrepräsentation die Türme von Links nach Rechts nach dem Schema `<Position des Turmes> <TAB><Stringrepräsentation des Turmes>` und durch Zeilenumbrüche getrennt zurück. Leerzeichen, Tabs, Zeilenumbrüche etc. am Ende des Strings sind zu entfernen.

Beispiel:

LEFT 7,5,1

MIDDLE 6,3

RIGHT 4,2

Implementieren Sie die folgende MainMethode:

- `public static void main(String[] args)`

Diese Methode soll die Funktionalität Ihres Programmes demonstrieren. Erstellen Sie dazu ein Spiel mit einer

von Ihnen gewählten Größe (der Algorithmus hat eine exponentielle Laufzeit, nehmen Sie also kein zu großes Spiel - ein kleiner zweistelliger Wert ist vollkommen ausreichend). Geben Sie dessen Zustand auf die Konsole aus. Anschließend lösen Sie das Spiel und geben den Zustand erneut aus.

Zugehörige Testklasse: GameTest

Viel Erfolg!